To proceed with the setup, execute the following command to install Jest into your project.

```
npm install --save-dev jest
```

Using Jest's `expect` assertions (e.g., on line 73), you can validate that a variable or response matches an expected value. This comparison determines whether the test passes or fails.

```
const { handles } = require('./queryAsset.js');
const axios = require('axios');
const { getToken } = require('./oauth.js');


jest.mock('axios');
jest.mock('./oauth');

describe('handles function', () => {
  const mockContext = {
    log: jest.fn(),
    error: jest.fn()
  };

  const mockRequest = (headers = {}, query = {}) => ({
    headers: {
      get: (key) => headers[key]
    },
    query: {
      get: (key) => query[key]
    }
  });

  const mockData = {
    value: [{

"_crf7f_ois_asset_dat_itemlookup_value@OData.Community.Display.V1.FormattedValue": "Asset A",
      "crf7f_phone_numbers": "123-456-7890",
      "crf7f_os_version": "Windows 11"
    }]
  };
```

```
  beforeEach(() => {
    jest.clearAllMocks();
    process.env.DATAVERSE_URL = 'https://example.crm.dynamics.com';
  });

  it('returns 400 if client secret is missing', async () => {
    const response = await handles(mockRequest({}, { id: 'abc' }),
mockContext);
    expect(response.status).toBe(400);
    expect(response.jsonBody.error).toBe('Missing client secret');
  });

  it('returns 403 if token is invalid', async () => {
    getToken.mockResolvedValue(null);
    const response = await handles(mockRequest({ 'x-client-secret':
'secret' }, { id: 'abc' }), mockContext);
    expect(response.status).toBe(403);
    expect(response.jsonBody.error).toBe('Unauthorized');
  });

  it('returns 404 if no data is found', async () => {
    getToken.mockResolvedValue('valid-token');
    axios.get.mockResolvedValue({ data: { value: [] } });

    const response = await handles(mockRequest({ 'x-client-secret':
'secret' }, { id: 'abc' }), mockContext);
    expect(response.status).toBe(404);
    expect(response.jsonBody.details).toBe('No matching records
found');
  });

  it('returns 200 with filtered data', async () => {
    getToken.mockResolvedValue('valid-token');
    axios.get.mockResolvedValue({ data: mockData });

    const response = await handles(mockRequest({ 'x-client-secret':
'secret' }, { id: 'abc' }), mockContext);
    expect(response.status).toBe(200);
```

```
    expect(response.jsonBody).toEqual({
      assetName: 'Asset A',
      phone: '123-456-7890',
      osVersion: 'Windows 11'
    });
  });

  it('returns 500 on axios error', async () => {
    getToken.mockResolvedValue('valid-token');
    axios.get.mockRejectedValue(new Error('Network error'));

    const response = await handles(mockRequest({ 'x-client-secret':
'secret' }, { id: 'abc' }), mockContext);
    expect(response.status).toBe(500);
    expect(response.jsonBody.error).toBe('Dataverse query failed');
  });
});
```

To execute tests, you have the option to run them either locally or remotely. For local execution, use the command `npm test`. For remote execution via GitHub Workflows, you can define a workflow by creating a `.yml` file within the `.github/workflows` directory

Locally:
npm test -- --coverage

```
PS C:\Users\MatthewHung\repos\itam-api-documentation\azure-functions> npm test
> jest

 FAIL  ./queryAssets.test.js
  ● Test suite failed to run

    Jest encountered an unexpected token

    Jest failed to parse a file. This happens e.g. when your code or its dependencies use non-standard JavaScript syntax, or when Jest is n
ot configured to support such syntax.

    Out of the box Jest supports Babel, which will be used to transform your files into valid JS based on your Babel configuration.
```

```
Details:

C:\Users\MatthewHung\repos\itam-api-documentation\azure-functions\queryAsset.js:1
import { app } from '@azure/functions';
^^^^^^

SyntaxError: Cannot use import statement outside a module

  1 |
> 2 | const { handles } = require('./queryAsset.js');
    |                     ^
```

When running tests locally, any failures will be reported directly in the terminal, providing detailed output to help identify and troubleshoot issues.

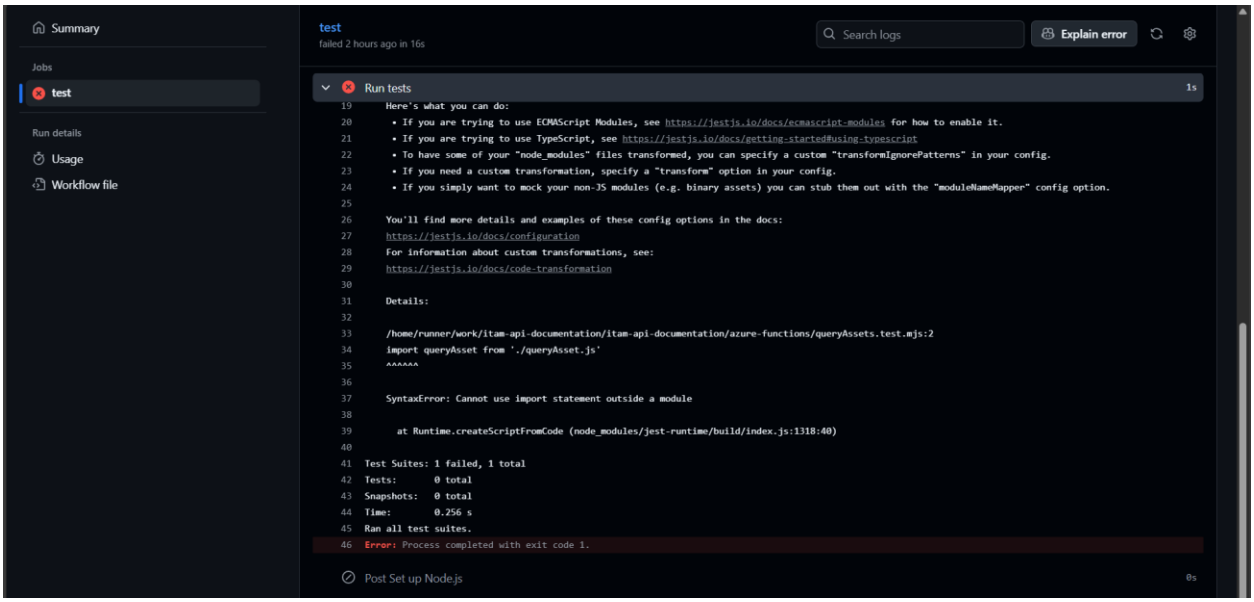
To enable remote test execution via GitHub, you must first configure a workflow by creating a `.yml` file within the `.github/workflows` directory of your repository.

```yaml
name: Azure Function Test

on:
  push:
    branches:
      - main
      - automated-testing-for-api-endpoints-AT-114

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: "18"
      - name: Install dependencies
        run: npm install
      - name: Run tests
        run: npm test
```

Next, ensure that Jest is defined under the `scripts` section of your `package.json` file. This allows you to run tests using a simple command such as `npm test`

```
"scripts": {
  "start": "func start",
  "test": "jest"
}
```

The behavior of your GitHub Actions workflow is determined by the value specified in the on property within the `.yml` file. This property defines the event(s) that will trigger the workflow, such as a push to a branch or a pull request



When running Github Actions, the test action will automatically run on a github hosted runner. If cost prohibitive, the action can be ran on a azure vm. [Adding self-hosted runners - GitHub Docs]

PLAN

implement a workflow to test our code before pushing to main. Before pushing, local tests must be done to ensure code is passing tests.
Test Types:

- **Input validation**: Test with valid and invalid payloads. (missing parameters)
- **Business logic**: Ensure core logic behaves correctly. (able to call child functions)
- **Output format**: Validate the structure and content of the response. (validate output structure of api)
- **Error handling**: Simulate failures and ensure proper error responses. (validate correct error codes)

- Integration: Validate writing to Dataverse.

Naming convention. When making new tests, each JavaScript file, will have their own tests stored in their own respective files. Naming convention will execute as follows: {filename}.test.js. Files will be stored in same directory as their respective .js file. If both local and remote tests are passed, code is able to merge. On this server, we can set local environment variable as well. You can store up to 1,000 organization variables, 500 variables per repository, and 100 variables per environment.